

Reach-Avoid Differential Game for UAV Interception

Implementation Documentation

Based on the paper by Bui, Monckton, and Chen
*“Reach-Avoid Differential Game with Reachability Analysis
for UAVs — A Decomposition Approach”*
(arXiv:2512.22793)

Framework:	Hamilton-Jacobi Reachability Analysis
Solver:	OptimizedDP-compatible API (hj_reachability v0.7.0 backend)
Simulation:	Gazebo Harmonic + PX4 SITL
Hardware:	Crazyflie 2.1 + Crazyswarm2
Middleware:	ROS 2 Humble
Version:	0.3.0

March 23, 2026

Contents

1	Project Overview	3
1.1	Key Results	3
2	Mathematical Foundation	3
2.1	Dynamics Modeling	3
2.2	Game Decomposition	4
2.3	Hamilton-Jacobi Reachability	4
2.4	Invariant Capture Sets	4
2.5	Computation Pipeline	5
2.6	Capture Guarantee Analysis	5
3	Implementation Architecture	5
3.1	Repository Structure	5
3.2	Offline Computation Package: <code>reach_avoid_game</code>	6
3.2.1	OptimizedDP-Compatible Solver API (<code>odp/</code>)	6
3.2.2	Dynamics Classes	6
3.2.3	Solver Functions	7
3.2.4	Control Extraction	8
3.2.5	Winning Conditions	8
3.2.6	Value Function I/O	8
3.3	Online Control: ROS 2 Workspace	9
3.3.1	Package Overview	9
3.3.2	Defender Controller Node	9
3.3.3	Attacker Controller Node	10
3.3.4	ROS 2 Topic Architecture	11
3.4	Configuration	11
4	Running the Offline Computation	12
4.1	Prerequisites	12
4.2	Computing Value Functions	12
4.2.1	Step 1: Vertical Pipeline	12
4.2.2	Step 2: Horizontal Pipeline	13
4.3	Running Numerical Simulations	13
4.4	Generating Paper Figures	13
4.5	Running Tests	13
5	Running the Gazebo Simulation	14
5.1	Quick Start (3 Commands)	14
5.2	What Happens at Startup	14
5.3	Monitoring the Game	15
5.4	Expected Game Behavior	16
5.5	Attacker Control Modes	16
5.6	Architecture Details	17
5.6.1	Docker Image	17
5.6.2	Gazebo World	17
5.6.3	PX4 SITL Configuration	17
5.6.4	Ground Truth Relay	17
5.6.5	PX4 Adapter	18
5.7	Kinematic Simulation Mode (No PX4)	18

6	Running on Real Drones	18
6.1	Hardware Requirements	18
6.2	Software Setup	19
6.3	Crazyflie Configuration	19
6.4	Pre-Flight Checklist	19
6.5	Launch Sequence	20
6.6	Safety Monitor	20
6.7	Crazyswarm2 Adapter	20
6.8	Tuning for Real Hardware	21
7	End-to-End Simulation Results	21
7.0.1	Scripted Attacker Scenarios	21
7.0.2	Optimal Attacker Scenario	22
7.0.3	Optimal Attacker Behavior Validation	22
8	Troubleshooting	22
9	References	24

Project Overview

This project implements a **3D reach-avoid differential game** between two unmanned aerial vehicles (UAVs). One drone, the **defender**, attempts to intercept a second drone, the **attacker**, before the attacker reaches a designated target region. The attacker simultaneously tries to navigate to its goal while avoiding both obstacles and the defender.

The core challenge is that computing optimal strategies for two drones in 3D space would require solving a Hamilton-Jacobi (HJ) partial differential equation on a **24-dimensional** joint state space (12 dimensions per quadrotor). This is computationally intractable. The paper introduces a **decomposition framework** that splits the problem into two solvable sub-games:

1. **Horizontal sub-game (6D):** Models the x-y plane dynamics of both drones, including obstacles and the target region.
2. **Vertical sub-game (3D):** Models the z-axis dynamics of both drones.

Each sub-game is solved independently using HJ reachability analysis, and the solutions are recombined using a novel **reach-track control algorithm** that provides provable capture guarantees.

The implementation spans three layers:

- **Offline computation:** Value functions are solved on a discretized grid using an OptimizedDP-compatible solver API (backed by `hj_reachability` with JAX/GPU acceleration). These represent optimal strategies for all possible states.
- **Online control:** Real-time ROS 2 controllers for both drones. The **defender** reads drone states, interpolates precomputed value functions, extracts optimal controls via gradient computation, and publishes velocity commands at 50 Hz (with PID pursuit fallback outside the winning region). The **attacker** uses game-theoretic optimal control by default, extracting bang-bang velocity commands from the same value functions to optimally evade the defender while navigating toward the target.
- **Simulation & hardware:** The game runs in Gazebo with PX4 autopilot for physics-based validation, as a lightweight kinematic simulation without PX4, or on real Crazyflie 2.1 drones with motion capture for state estimation.

Key Results

- The decomposition reduces the problem from 24D (intractable) to a 6D + 3D pair (solvable in minutes to hours).
- Formal capture guarantees: if the initial state is in the defender's **winning region**, the defender is guaranteed to capture the attacker regardless of the attacker's strategy.
- The solution is **conservative**: the decomposition's winning region is a subset of the true optimal winning region, meaning the defender never falsely claims it can win.

Mathematical Foundation

Dynamics Modeling

Both drones are simplified from the full 12D quadrotor dynamics:

Definition 1 (Defender — Double Integrator). *The defender has inertia (cannot change velocity instantaneously):*

$$\dot{x}_D = v_D, \quad \dot{v}_D = K(u - v_D) \quad (1)$$

where u is the velocity command and $K = \text{diag}(k_x, k_y, k_z)$ are proportional gains. The state is 6D: position (x_D, y_D, z_D) and velocity $(v_{D,x}, v_{D,y}, v_{D,z})$.

Definition 2 (Attacker — Single Integrator). *The attacker has no inertia (controls velocity directly):*

$$\dot{x}_A = d \quad (2)$$

where d is the attacker's velocity command. The state is 3D: position (x_A, y_A, z_A) . This **over-estimates** the attacker's capability, making the defender's strategy more conservative.

Speed constraints:

$$\|(u_x, u_y)\| \leq U_D^h = 6 \text{ m/s}, \quad \|(d_x, d_y)\| \leq U_A^h = 3 \text{ m/s} \quad (3)$$

$$|u_z| \leq U_D^z = 4 \text{ m/s}, \quad |d_z| \leq U_A^z = 2 \text{ m/s} \quad (4)$$

The defender is twice as fast as the attacker in all directions.

Game Decomposition

The capture condition is decomposed from a sphere into a **cylinder**:

- **Original:** $\|p_A - p_D\|_2 \leq d_c$ (spherical capture zone)
- **Decomposed:**

$$\textbf{Horizontal: } \sqrt{(x_A - x_D)^2 + (y_A - y_D)^2} \leq d_h = 3 \text{ m} \quad (5)$$

$$\textbf{Vertical: } |z_A - z_D| \leq d_z = 1 \text{ m} \quad (6)$$

Key assumption: The target region $\mathcal{T}$ and obstacles depend only on horizontal (x, y) coordinates, not altitude. This is practical for airspace corridor scenarios.

Hamilton-Jacobi Reachability

For each sub-game, we solve the HJ variational inequality backward in time:

$$\min \left\{ \frac{\partial V}{\partial t} + H(x, \nabla_x V), V(x, t) - l(x) \right\} = 0 \quad (7)$$

where the Hamiltonian is:

$$H(x, p) = \max_u \min_d p \cdot f(x, u, d) \quad (8)$$

The defender maximizes (tries to capture) and the attacker minimizes (tries to escape). The **zero level set** $\{x : V(x) \leq 0\}$ defines the winning region.

Invariant Capture Sets

Simply combining the two sub-game solutions does not guarantee simultaneous capture in both directions. The defender might overshoot vertically due to inertia while capturing horizontally. The solution uses **invariant capture sets**:

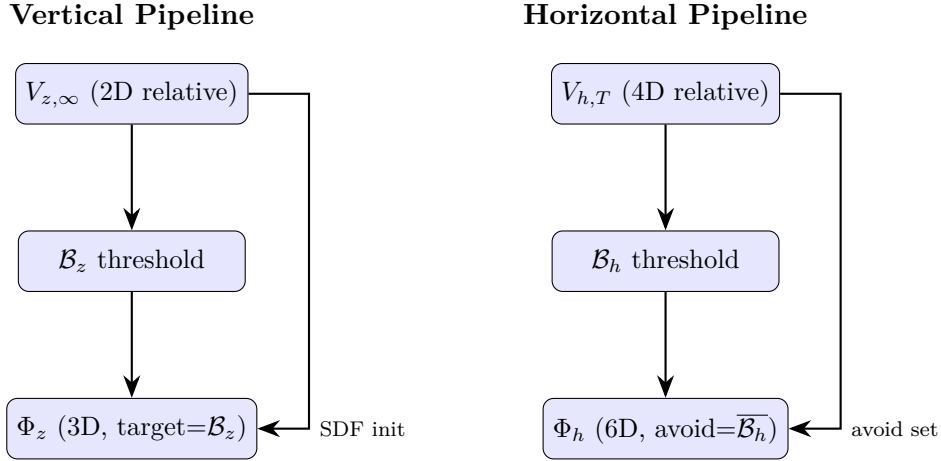
Definition 3 (Invariant Capture Set $\mathcal{B}_z$). $\mathcal{B}_z = \{x : V_{z,\infty}(x) \leq d_z\}$ where $V_{z,\infty}$ is the maximum distance value function. Once inside $\mathcal{B}_z$, the defender can keep $|z_D - z_A| \leq d_z$ **forever** using a tracking controller.

Definition 4 (Invariant Capture Set $\mathcal{B}_h$). $\mathcal{B}_h = \{x : V_{h,T}(x) \leq d_h\}$ where $V_{h,T}$ is computed with $T = 2.5$ s (does not converge as $T \rightarrow \infty$ due to the horizontal dynamics structure).

These feed back into the reach-avoid computation: Φ_z uses $\mathcal{B}_z$ as its target set (Eq. 38 of the paper), and Φ_h uses the complement of $\mathcal{B}_h$ as part of its avoid set (Eq. 39).

Computation Pipeline

The correct computation order (matching the paper) is:



Capture Guarantee Analysis

Theorem 1 (Defender Wins). *The defender is guaranteed to capture the attacker if all of the following hold:*

1. $x^z \in \mathcal{W}_{D,z}$ — vertical state is in defender's vertical winning region
2. $T_{\text{goal}} > T_{\text{capture}}$ — defender captures vertically before attacker reaches goal
3. $x^h \in \mathcal{W}_{D,h}$ — horizontal state is in defender's horizontal winning region

Where T_{goal} is the earliest time the attacker can reach its target (from $\Phi_{A,\text{reach}}$) and T_{capture} is the earliest time the defender achieves vertical capture (from Φ_z time slices).

Implementation Architecture

Repository Structure

Top-Level Directory Layout

```

/workspace/
  config/
    game_params.yaml      # Game parameters and grid presets
  data/
    value_functions/      # Precomputed .npz value function files
    plots/paper_figures/  # Generated paper figures
    simulations/           # Simulation trajectory outputs
  reach_avoid_game/      # Core Python package (offline computation)
    src/reach_avoid_game/
      config.py           # GameConfig dataclass
      dynamics/           # HJ dynamics classes (dual API)

```

```

solvers/          # HJ solvers and utilities
odp/              # OptimizedDP-compatible API layer
  grid.py         # Grid class (NumPy, matches ODP API)
  solver.py       # HJSolver wrapper + computeSpatDerivArray
  shapes.py       # Shape functions for initial values
  visualization/  # Plotting functions
scripts/         # CLI entry points
tests/           # Pytest test suite (60 tests)
tests/
  integration/
    test_kinematic_game.py # Standalone end-to-end simulation
reach_avoid_ws/   # ROS 2 workspace (online control)
src/
  reach_avoid_bringup/ # Launch files and parameters
  reach_avoid_sim/     # Gazebo adapters, PX4 bridge
  reach_avoid_controller/ # Defender reach-track controller
  attacker_controller/  # Attacker control modes
  reach_avoid_viz/      # RViz MarkerArray visualization
  reach_avoid_hw/       # Crazyflie hardware interface

```

Offline Computation Package: reach_avoid_game

3.2.1 OptimizedDP-Compatible Solver API (odp/)

A custom OptimizedDP-compatible package provides the paper’s solver API without requiring HeteroCL (which is only available for Python 3.8 via conda). The package includes:

- `grid.py`: `Grid(minBounds, maxBounds, dims, pts_each_dim, periodicDims)` — same API as `odp.Grid.GridProcessing.Grid` from the SFU-MARS OptimizedDP repository.
- `solver.py`: `HJSolver(dynamics, grid, values, tau, compMethod)` — OptimizedDP-compatible entry point that delegates to `hj_reachability`’s proven Lax-Friedrichs solver internally. Also provides `computeSpatDerivArray(grid, V, dim)` for spatial derivative extraction.
- `shapes.py`: `CylinderShape`, `ShapeRectangle` for constructing initial value function SDFs.

The `HJSolver` translates OptimizedDP’s `compMethod` dictionary to `hj_reachability` solver settings:

Table 1: Mapping between OptimizedDP and `hj_reachability` solver modes.

OptimizedDP compMethod	hj_reachability equivalent
"TargetSetMode": "minVWithV0"	Backward Reachable Tube ($V = \min(V, V_0)$)
"TargetSetMode": "maxVWithV0"	Max-over-time ($V = \max(V, V_0)$)
"ObstacleSetMode": "maxVWithObstacle"	Reach-avoid with obstacle constraint
"TargetSetMode": "none"	Pure backward reachable set

3.2.2 Dynamics Classes

Each dynamics class inherits from `hj.ControlAndDisturbanceAffineDynamics` (for the HJ solver) and additionally provides OptimizedDP-style pure-Python methods for online control extraction. The class implements three `hj_reachability` methods:

Each class provides the **hj_reachability interface** (for the solver):

Table 2: Dynamics classes and their state spaces.

Class	Dim	State	File
VerticalGameDynamics	3D	$(z_D, v_{D,z}, z_A)$	vertical_game.py
VerticalRelativeDynamics	2D	$(z_{\text{rel}}, v_{D,z})$	vertical_relative.py
HorizontalGameDynamics	6D	$(x_D, y_D, v_{D,x}, v_{D,y}, x_A, y_A)$	horizontal_game.py
HorizontalRelativeDynamics	4D	$(x_{\text{rel}}, y_{\text{rel}}, v_{D,x}, v_{D,y})$	horizontal_relative.py
HorizontalGameTrackingDynamics	6D	same as above	horizontal_game.py
AttackerReachingDynamics	2D	(x_A, y_A)	attacker_reaching.py

- `open_loop_dynamics(state, time)` — drift dynamics $f(x)$ when $u = 0$, $d = 0$
- `control_jacobian(state, time)` — matrix G_u mapping control to state derivatives
- `disturbance_jacobian(state, time)` — matrix G_d mapping disturbance to state derivatives

Key classes also provide the **OptimizedDP-compatible NumPy methods** (for online control extraction):

- `opt_ctrl_numpy(state, spat_deriv)` — optimal control from value function gradient
- `opt_dstb_numpy(state, spat_deriv)` — optimal disturbance from gradient
- `dynamics_numpy(state, u, d)` — state derivatives given control and disturbance

The `HorizontalGameTrackingDynamics` class is identical to `HorizontalGameDynamics` except with **swapped optimization**: `control_mode="min"` (defender minimizes distance) and `disturbance_mode="max"` (attacker maximizes distance). This is used for computing the maximum distance value function $V_{h,T}$ in 6D coordinates with obstacle penalties.

3.2.3 Solver Functions

All solvers are in the `solvers/` subdirectory:

Table 3: Value functions computed by each solver function.

Function	Output	Grid	Description
<code>solve_vertical_max_distance</code>	$V_{z,\infty}$	2D	Max distance tracking (converged)
<code>compute_invariant_set_Bz</code>	$\mathcal{B}_z$	2D	Threshold $V_{z,\infty} \leq d_z$
<code>solve_vertical_reach_avoid</code>	Φ_z	3D	Vertical reach-avoid with $\mathcal{B}_z$ target
<code>solve_horizontal_max_distance</code>	$V_{h,T}$	4D	Max distance tracking ($T = 2.5$ s)
<code>compute_invariant_set_Bh</code>	$\mathcal{B}_h$	4D	Threshold $V_{h,T} \leq d_h$
<code>solve_attacker_reaching</code>	$\Phi_{A,\text{reach}}$	2D	Attacker time-to-goal
<code>solve_horizontal_reach_avoid</code>	Φ_h	6D	Horizontal reach-avoid with $\overline{\mathcal{B}_h}$ avoid
<code>solve_horizontal_max_distance_6d</code>	$V_{h,T}^{6D}$	6D	6D extension with obstacles

3.2.4 Control Extraction

Optimal controls are extracted from value function gradients using central finite differences:

$$\nabla V(x) \approx \left[\frac{V(x + h e_i) - V(x - h e_i)}{2h} \right]_{i=1}^n \quad (9)$$

The defender's optimal control (maximizing the value function):

$$u_z^* = U_{D,z} \cdot \text{sign} \left(\frac{\partial V}{\partial v_{D,z}} \cdot k_z \right) \quad (10)$$

The attacker's optimal disturbance (minimizing the value function):

$$d_z^* = -U_{A,z} \cdot \text{sign} \left(\frac{\partial V}{\partial z_A} \right) \quad (11)$$

Value function interpolation uses `scipy.interpolate.RegularGridInterpolator` with linear interpolation and appropriate boundary handling (`fill_value` set to large positive values for out-of-domain queries).

3.2.5 Winning Conditions

The `winning_conditions.py` module implements:

- `compute_T_goal`: Estimates earliest time the attacker reaches its target from $\Phi_{A,\text{reach}}$.
- `compute_T_capture`: Estimates vertical capture time from Φ_z magnitude.
- `compute_T_capture_from_slices`: The paper-accurate method — iterates through saved Φ_z time slices and finds the first time at which $\Phi_z(x, t) \leq 0$.
- `check_defender_wins`: Combined check of all winning conditions.
- `get_winning_regions`: Extracts $\mathcal{W}_D$ and $\mathcal{W}_A$ masks from a value function.

3.2.6 Value Function I/O

All value functions are stored as NumPy `.npz` files with a standardized format:

```
@dataclass
class ValueFunctionData:
    values: np.ndarray      # The value function array
    grid_min: np.ndarray    # Grid lower bounds
    grid_max: np.ndarray    # Grid upper bounds
    grid_shape: tuple       # Grid dimensions
    params: dict            # Game parameters used
    description: str        # Human-readable description
```

Time slices (for T_{capture} extraction) are stored separately as `phi_z_time_slices.npz` containing the full time evolution of Φ_z .

Table 4: ROS 2 packages and their roles.

Package	Purpose
reach_avoid_bringup	Launch files, configuration YAML, dependency management
reach_avoid_sim	PX4 adapter (MAVLink $\leftrightarrow$ ROS 2), ground truth relay from Gazebo
reach_avoid_controller	Defender controller implementing Algorithms 1 & 2 with value function lookup
attacker_controller	Attacker with modes: optimal (default, HJ bang-bang control), scripted waypoints, keyboard teleop, switchable
reach_avoid_viz	MarkerArray publisher for obstacles, target, capture zones in RViz
reach_avoid_hw	Crazyswarm2 adapter for Crazyflie drones, safety monitor

Online Control: ROS 2 Workspace

3.3.1 Package Overview

3.3.2 Defender Controller Node

The `DefenderControllerNode` runs at 50 Hz and implements the two-algorithm reach-track controller:

Algorithm 1 Vertical Reach-Track (Algorithm 1) — with winning region guard

```

1: Input: Vertical state  $(z_D, v_{D,z}, z_A)$ , value functions  $\Phi_z, V_{z,\infty}, \mathcal{B}_z$ 
2: Compute  $z_{\text{rel}} \leftarrow z_D - z_A$ 
3:  $in\_winning \leftarrow \Phi_z(z_D, v_{D,z}, z_A) \leq 0$  ▷ Check defender's winning region
4: Look up  $\mathcal{B}_z(z_{\text{rel}}, v_{D,z})$ 
5: if NOT in  $\mathcal{B}_z$  then
6:   if  $in\_winning$  then
7:      $u_z \leftarrow$  optimal reaching control from  $\nabla \Phi_z$  ▷ Mode: REACHING
8:   else
9:      $u_z \leftarrow$  PID( $z_A - z_D$ ) ▷ Mode: PID_PURSUIT
10:  end if
11: else if deep inside  $\mathcal{B}_z$  then
12:    $u_z \leftarrow$  PID( $z_A - z_D$ ) ▷ Mode: PID_DEEP
13: else
14:    $u_z \leftarrow$  optimal tracking control from  $\nabla V_{z,\infty}$  ▷ Mode: TRACKING
15: end if
16: return  $u_z$ 

```

Note

The **winning region guard** ($\Phi \leq 0$ check) is critical for robustness. The paper's optimal HJ controls are only valid inside the winning region. On coarse grids (e.g., the `dev` preset with a $9 \times 7 \times 5 \times 5 \times 9 \times 7$ 6D grid), $\Phi_h > 0$ everywhere, meaning value function gradients point in arbitrary directions. PID pursuit provides a reliable fallback that always drives the defender toward the attacker.

Algorithm 2 Horizontal Reach-Track-Avoid (Algorithm 2) — with winning region guard

```

1: Input: Horizontal state  $(x_D, y_D, v_{D,x}, v_{D,y}, x_A, y_A)$ , value functions  $\Phi_h, V_{h,T}, \mathcal{B}_h$ 
2:  $in\_winning \leftarrow \Phi_h(x_D, y_D, v_{D,x}, v_{D,y}, x_A, y_A) \leq 0$  ▷ Check winning region
3: Compute relative state  $(x_{rel}, y_{rel}, v_{D,x}, v_{D,y})$ 
4: Look up  $\mathcal{B}_h(x_{rel}, y_{rel}, v_{D,x}, v_{D,y})$ 
5: if NOT in  $\mathcal{B}_h$  then
6:   if  $in\_winning$  then
7:      $(u_x, u_y) \leftarrow$  optimal reaching control from  $\nabla\Phi_h$  ▷ Mode: REACHING
8:   else
9:      $(u_x, u_y) \leftarrow$  PID( $x_A - x_D, y_A - y_D$ ) ▷ Mode: PID_PURSUIT
10:  end if
11: else if deep inside  $\mathcal{B}_h$  then
12:    $(u_x, u_y) \leftarrow$  PID( $x_A - x_D, y_A - y_D$ ) ▷ Mode: PID_DEEP
13: else
14:    $(u_x, u_y) \leftarrow$  optimal tracking control from  $\nabla V_{h,T}$  ▷ Mode: TRACKING
15: end if
16: return  $(u_x, u_y)$ 

```

3.3.3 Attacker Controller Node

The **AttackerControllerNode** runs at 20 Hz and supports multiple control modes. The default mode is **optimal**, which implements game-theoretic bang-bang control using Hamilton-Jacobi value function gradients.

Optimal Control Mode. The attacker acts as the **disturbance** player in the HJ game formulation, minimizing the value function to optimally evade the defender while heading toward the target. The control law is bang-bang (always at maximum speed):

$$d_x^* = -U_A^h \cdot \text{sign}\left(\frac{\partial\Phi_h}{\partial x_A}\right), \quad d_y^* = -U_A^h \cdot \text{sign}\left(\frac{\partial\Phi_h}{\partial y_A}\right) \quad (12)$$

$$d_z^* = -U_A^z \cdot \text{sign}\left(\frac{\partial\Phi_z}{\partial z_A}\right) \quad (13)$$

where:

- Φ_h is the 6D horizontal game value function with state $[x_D, y_D, v_{D,x}, v_{D,y}, x_A, y_A]$
- Φ_z is the 3D vertical game value function with state $[z_D, v_{D,z}, z_A]$
- $U_A^h = 3.0$ m/s (max horizontal speed), $U_A^z = 2.0$ m/s (max vertical speed)
- Gradients at indices 4, 5 (for x_A, y_A) and index 2 (for z_A) respectively

The negative sign reflects the attacker's role as the **minimizer** in the game (opposite to the defender, who maximizes). This ensures the attacker moves in the direction that most decreases the value function, corresponding to the worst-case disturbance from the defender's perspective.

Fallback Behavior. If the value function gradient is near-zero (e.g., at grid boundaries) or the defender's state is not yet available, the attacker falls back to **simple goal-seeking**: a unit direction vector toward the target center (41.5, 12.5) scaled to U_A^h , with proportional altitude control toward the target altitude.

State Subscriptions. In optimal mode, the attacker subscribes to the defender's state in addition to its own:

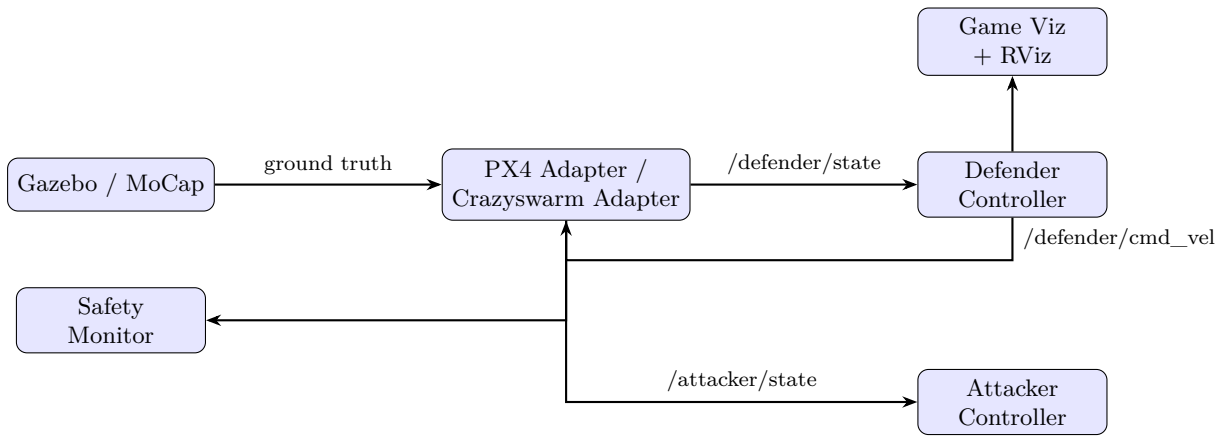
- `/attacker/state`, `/attacker/velocity` — own position and velocity
- `/defender/state`, `/defender/velocity` — defender position and velocity (required for the 6D Φ_h and 3D Φ_z lookups)

This matches the paper's **perfect information** assumption: both players know each other's full state.

Other Modes.

- **scripted** — follows predefined waypoints toward target at constant speed
- **keyboard** — manual control via `teleop_twist_keyboard`
- **switchable** — start in scripted, switch at runtime via `/attacker/set_mode` service

3.3.4 ROS 2 Topic Architecture



Key topics:

- `/defender/state` (PoseStamped) — defender position from ground truth
- `/attacker/state` (PoseStamped) — attacker position
- `/defender/cmd_vel` (Twist) — velocity command to PX4/Crazyflie
- `/game/status` (String) — game status and mode, e.g. DEFENDER_WINNING | z:tracking
h:pid_pursuit | d_h=4.32 d_z=0.12
- `/safety/emergency` (Bool) — emergency landing trigger

Configuration

All game parameters are defined in `config/game_params.yaml`:

Key parameters from game_params.yaml

```

room:
  x_min: 0.0    x_max: 45.0
  y_min: 0.0    y_max: 25.0
  z_min: 0.0    z_max: 20.0
defender:
  max_speed_horizontal: 6.0  # m/s
  max_speed_vertical: 4.0    # m/s
  k_x: 0.7    k_y: 0.7    k_z: 1.5
attacker:

```

```

max_speed_horizontal: 3.0    # m/s
max_speed_vertical: 2.0     # m/s
capture:
  d_h: 3.0    # horizontal capture distance (m)
  d_z: 1.0    # vertical capture distance (m)
target_region:
  x_min: 38.0    x_max: 45.0
  y_min: 10.0    y_max: 15.0
obstacles:
  - type: box
    x_min: 15.0    x_max: 20.0
    y_min: 5.0     y_max: 20.0

```

Three grid presets are available:

Table 5: Grid resolution presets.

Preset	Vertical 2D	Vertical 3D	Horizontal 4D/6D
dev	51×31	$24 \times 10 \times 24$	$21^2 \times 11^2$
medium	120×50	$120 \times 50 \times 120$	$43^2 \times 6^2$
paper	240×100	$240 \times 100 \times 240$	$85^2 \times 8^2$

Running the Offline Computation

Prerequisites

```

# Install the reach_avoid_game package (editable mode)
cd /workspace/reach_avoid_game
pip install -e ".[dev]"

# Verify installation
python3 -c "import reach_avoid_game; print('OK')"
python3 -c "import hj_reachability; print('hj_reachability OK')"

```

Computing Value Functions

The computation must follow the correct dependency order.

4.2.1 Step 1: Vertical Pipeline

```

# Compute V_z_inf -> B_z -> Phi_z (with B_z feedback)
python3 /workspace/reach_avoid_game/scripts/compute_vertical.py \
  --preset dev

# Output files:
#   /workspace/data/value_functions/V_z_inf.npz
#   /workspace/data/value_functions/B_z.npz
#   /workspace/data/value_functions/phi_z.npz
#   /workspace/data/value_functions/phi_z_time_slices.npz

```

4.2.2 Step 2: Horizontal Pipeline

```
# Compute  $V_{h,T} \rightarrow B_h \rightarrow \phi_{A,reach} \rightarrow \Phi_h$  (with  $B_h$  feedback)
python3 /workspace/reach_avoid_game/scripts/compute_horizontal.py \
    --preset dev

# Optional: include 6D extension
python3 /workspace/reach_avoid_game/scripts/compute_horizontal.py \
    --preset dev --include-6d

# Output files:
# /workspace/data/value_functions/ $V_{h,T}$ .npz
# /workspace/data/value_functions/ $B_h$ .npz
# /workspace/data/value_functions/ $\phi_{A,reach}$ .npz
# /workspace/data/value_functions/ $\Phi_h$ .npz
# /workspace/data/value_functions/ $V_{h,T,6d}$ .npz (if --include-6d)
```

Note

The dev preset takes minutes. The medium preset takes tens of minutes. The paper preset (~3 hours for Φ_h alone) requires significant RAM and ideally a GPU.

Running Numerical Simulations

```
# Vertical-only simulation (Algorithm 1)
python3 /workspace/reach_avoid_game/scripts/numerical_sim.py \
    --mode vertical --T 10.0

# Combined 3D simulation (Algorithm 1 + Algorithm 2)
python3 /workspace/reach_avoid_game/scripts/numerical_sim.py \
    --mode combined --T 10.0

# Quick sanity check (1 step, verifies value function loading)
python3 /workspace/reach_avoid_game/scripts/numerical_sim.py \
    --mode combined --check-only
```

Generating Paper Figures

```
python3 /workspace/reach_avoid_game/scripts/generate_paper_figures.py \
    --output-dir /workspace/data/plots/paper_figures/ \
    --preset dev
```

This generates 10 figures corresponding to Figures 4–12 of the paper: $\mathcal{B}_z$ invariant set, Φ_z slices, vertical 3D view, $\mathcal{B}_h$ invariant set, Φ_h slices, winning regions, simulation trajectories, and a combined 3D game view.

Running Tests

```
cd /workspace/reach_avoid_game
python3 -m pytest tests/ -v

# 60 tests in reach_avoid_game + 31 in reach_avoid_controller = 91 total
# Covering:
# - Grid dimensions and value function shapes
```

```
# - Phi_z properties (monotonicity, sign conventions)
# - B_z non-emptiness and center containment
# - Control extraction within speed limits
# - Gradient finite-ness
# - T_capture from time slices
# - Winning region partitioning
# - Value function I/O round-trip
```

Running the Gazebo Simulation

Quick Start (3 Commands)

The entire simulation runs inside Docker. No local ROS 2 or Gazebo installation is needed — only Docker Desktop.

```
# 1. Build the image (first time takes ~20 min, cached afterwards)
docker build -f Dockerfile.sim -t reach-avoid-sim .

# 2. Start the simulation (auto-launches everything)
docker-compose -f docker-compose.sim.yml up

# 3. Verify drones are flying (in a separate terminal, after ~60s)
docker exec -it reach-avoid-sim bash -c \
'source /opt/ros/humble/setup.bash && \
source /home/simuser/ws/install/setup.bash && \
ros2 topic echo /attacker/state --field pose.position --once'
```

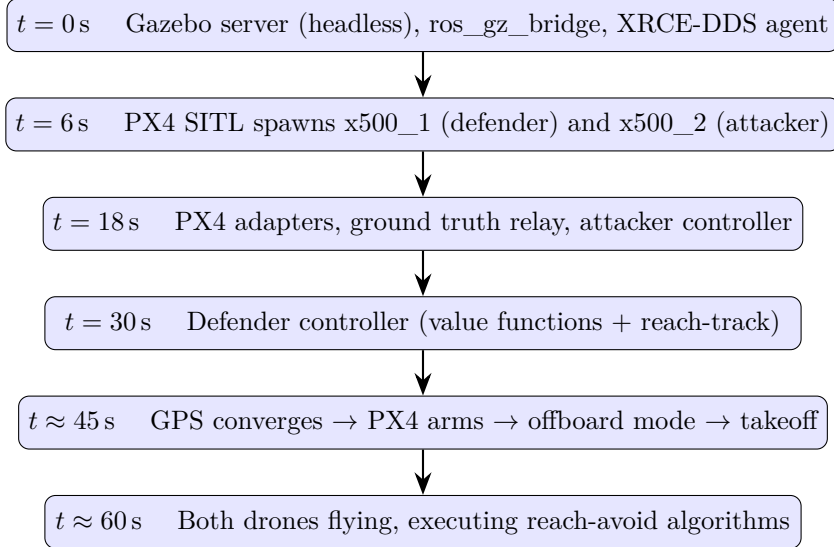
Expected output after ~60s: the attacker's position should be moving from its spawn (5, 20, 3) toward the target (41.5, 12.5, 10).

Warning

On macOS (Apple Silicon or Intel), Gazebo runs headless (no GUI window). The physics simulation, PX4, and all controllers run normally — you verify via `ros2 topic echo`. On Linux with a GPU and X11, Gazebo will open a 3D window.

What Happens at Startup

The container's CMD runs `ros2 launch reach_avoid_bringup full_game.launch.py`, which orchestrates the following startup sequence:



Monitoring the Game

All commands below require the ROS 2 environment. Use single quotes to avoid shell escaping issues:

```

# Shorthand: define a helper function (optional)
# rexec() { docker exec -it reach-avoid-sim bash -c \
#   "source /opt/ros/humble/setup.bash && \
#   source /home/simuser/ws/install/setup.bash && $1"; }

# --- Check which nodes are running ---
docker exec -it reach-avoid-sim bash -c \
  'source /opt/ros/humble/setup.bash && \
  source /home/simuser/ws/install/setup.bash && \
  ros2 node list'
# Expected: /attacker_controller /defender_controller
#           /ground_truth_relay /kinematic_sim or /px4_adapter_*
#           /game_viz /ros_gz_bridge /world_to_map_tf

# --- Watch attacker position (Ctrl+C to stop) ---
docker exec -it reach-avoid-sim bash -c \
  'source /opt/ros/humble/setup.bash && \
  source /home/simuser/ws/install/setup.bash && \
  ros2 topic echo /attacker/state --field pose.position'

# --- Watch defender position ---
docker exec -it reach-avoid-sim bash -c \
  'source /opt/ros/humble/setup.bash && \
  source /home/simuser/ws/install/setup.bash && \
  ros2 topic echo /defender/state --field pose.position'

# --- Check velocity commands ---
docker exec -it reach-avoid-sim bash -c \
  'source /opt/ros/humble/setup.bash && \
  source /home/simuser/ws/install/setup.bash && \
  ros2 topic echo /defender/cmd_vel --once'

# --- List all active topics ---
docker exec -it reach-avoid-sim bash -c \
  'source /opt/ros/humble/setup.bash && \

```



```
source /home/simuser/ws/install/setup.bash && \
ros2 topic list'

# --- Check PX4 arm/takeoff status ---
docker logs reach-avoid-sim 2>&1 | grep -iE "armed|takeoff"

# --- Stop the simulation ---
docker-compose -f docker-compose.sim.yml down
```

Expected Game Behavior

When the simulation is running correctly:

1. **Attacker** spawns at $(5, 20, 3)$ and uses **optimal game-theoretic control** by default:

- Loads Φ_h and Φ_z value functions, subscribes to defender state
- Extracts bang-bang controls from value function gradients (Eqs. 12–13)
- Flies at maximum speed: $U_A^h = 3.0$ m/s horizontal, $U_A^z = 2.0$ m/s vertical
- Actively evades the defender while navigating toward the target region
- Obstacle avoidance is implicit in the Φ_h value function (reach-avoid formulation)

If launched with `attacker_mode:=scripted`, it instead follows 6 waypoints at 1.6 m/s.

2. **Defender** spawns at $(5, 12.5, 3)$ and uses the **reach-track controller**:

- Loads precomputed value functions ($\Phi_z, V_{z,\infty}, \mathcal{B}_z, \Phi_h, V_{h,T}, \mathcal{B}_h$)
- Checks if current state is in the defender's **winning region** ($\Phi \leq 0$)
- If in winning region: computes optimal velocity via value function gradients (Algorithm 1 + 2)
- If outside winning region: uses **PID pursuit** — proportional control driving directly toward the attacker
- Velocity clamped to $U_D^h = 6.0$ m/s horizontal, $U_D^z = 4.0$ m/s vertical (matching game parameters)

3. **Game outcome**: the defender attempts to intercept the attacker before it reaches the target region $x \in [38, 45]$, $y \in [10, 15]$. Capture occurs when horizontal distance ≤ 3 m **and** vertical distance ≤ 1 m (cylindrical capture zone).

Attacker Control Modes

- **optimal (default)** — game-theoretic HJ bang-bang control using Φ_h and Φ_z gradients. The attacker minimizes the game value function, yielding the optimal evasion strategy from the paper. Requires precomputed value functions and defender state access.
- **scripted** — follows predefined waypoints toward target at constant speed (1.6 m/s)
- **keyboard** — manual control via `teleop_twist_keyboard`
- **switchable** — start in scripted, switch at runtime via `/attacker/set_mode` service

To override the default mode in any launch file:

```
ros2 launch reach_avoid_bringup full_game.launch.py attacker_mode:=scripted
```

Architecture Details

5.6.1 Docker Image

The `Dockerfile.sim` builds on Ubuntu 22.04 with:

- ROS 2 Humble Desktop + Gazebo Harmonic + `ros_gz` bridge
- PX4 Autopilot compiled for SITL with `gz_x500` quadrotor model
- micro-XRCE-DDS-Agent (PX4 ↔ ROS 2 bridge)
- `px4_msgs` and `px4_ros_com` for PX4 message types
- The full `reach_avoid_ws` workspace, pre-built with `colcon build`
- Value functions and game parameters copied from host

5.6.2 Gazebo World

The SDF world (`reach_avoid_arena.sdf`) defines:

- $45 \times 25 \times 20$ m room with four walls (matching `game_params.yaml`)
- Red obstacle: $x \in [15, 20]$, $y \in [5, 20]$ (center (17.5, 12.5, 10), size $5 \times 15 \times 20$ m)
- Green target region: $x \in [38, 45]$, $y \in [10, 15]$ (center (41.5, 12.5, 10), size $7 \times 5 \times 20$ m)
- Spherical coordinates (Zurich, 47.40°N, 8.55°E) for GPS sensor
- NavSat system plugin for GPS simulation
- DART physics at 1 kHz

5.6.3 PX4 SITL Configuration

Each PX4 instance runs with these parameters (set via `PX4_PARAM_*` environment variables):

Table 6: PX4 SITL parameter configuration.

Parameter	Value	Purpose
EKF2_GPS_CTRL	7	GPS fusion (position + velocity + altitude)
EKF2_MAG_TYPE	1	Automatic magnetometer fusion
EKF2_MAG_CHECK	0	Disable mag consistency check
COM_ARM_WO_GPS	1	Allow arming before GPS lock
CBRK_SUPPLY_CHK	894281	Bypass power supply check
CBRK_FLIGHTTERM	121212	Bypass flight termination
COM_DISARM_PRFLT	-1	Disable auto preflight disarm
FD_FAIL_P/R	0	Disable pitch/roll failure detection
COM_IMB_PROP_ACT	0	Disable propeller imbalance action

5.6.4 Ground Truth Relay

The ground truth relay node bridges PX4 local position to game coordinates:

- Subscribes to PX4's `VehicleLocalPosition` via `px4_msgs` (50 Hz)
- Converts NED (North-East-Down) to ENU (East-North-Up) by swapping $x \leftrightarrow y$ and negating z
- Adds spawn position offsets: defender (5, 12.5, 3), attacker (5, 20, 3)
- Publishes to `/defender/state` and `/attacker/state` as `PoseStamped`

5.6.5 PX4 Adapter

The PX4 adapter translates velocity commands to PX4 offboard control:

- Subscribes to `/defender/cmd_vel` (or `/attacker/cmd_vel`)
- Clamps velocities to $U_D^h = 6.0$ m/s horizontal, $U_D^z = 4.0$ m/s vertical (matching game parameters)
- Converts ENU to NED and publishes `TrajectorySetpoint + OffboardControlMode`
- Auto-engages offboard mode and force-arms PX4 after GPS convergence (~ 15 s)

Kinematic Simulation Mode (No PX4)

For faster iteration or environments where PX4 SITL is problematic, a lightweight kinematic simulator is available:

```
# In docker-compose.sim.yml, change the command to:
command: ros2 launch reach_avoid_bringup kinematic_game.launch.py

# Or run directly inside the container:
ros2 launch reach_avoid_bringup kinematic_game.launch.py
```

This mode replaces Gazebo + PX4 with simple velocity integration ($\mathbf{x}_{t+1} = \mathbf{x}_t + \mathbf{v} \cdot \Delta t$). The reach-avoid controllers are identical — only the physics layer changes. Startup is instant (no GPS convergence wait).

Running on Real Drones

Safety Critical

Hardware deployment involves real flying robots. Always have a safety pilot ready. Always test with conservative speed limits first. The safety monitor will trigger emergency landing on any violation, but human oversight is essential.

Hardware Requirements

Table 7: Required hardware components.

Component	Details
Crazyflie 2.1 (x2)	One defender, one attacker. Must have unique radio addresses.
Crazyradio PA (x1)	USB radio dongle for communicating with Crazyflies.
Motion capture system	OptiTrack, Vicon, or similar. Must publish pose at ≥ 100 Hz.
Ground station PC	Ubuntu 22.04, ROS 2 Humble, Crazyswarm2 installed.
Flying arena	Minimum $4 \times 4 \times 3$ m clear space with safety nets.

Software Setup

```
# 1. Install ROS 2 Humble (if not already installed)
sudo apt install ros-humble-desktop

# 2. Install CrazySwarm2
mkdir -p ~/crazySwarm2_ws/src
cd ~/crazySwarm2_ws/src
git clone https://github.com/IMRCLab/crazySwarm2
cd ..
colcon build --symlink-install
source install/setup.bash

# 3. Build the reach_avoid_ws workspace
cd /workspace/reach_avoid_ws
source /opt/ros/humble/setup.bash
colcon build --symlink-install
source install/setup.bash

# 4. Install the offline computation package
cd /workspace/reach_avoid_game
pip install -e .

# 5. Compute value functions (use medium or paper preset for real flights)
python3 scripts/compute_vertical.py --preset medium
python3 scripts/compute_horizontal.py --preset medium
```

Crazyflie Configuration

Edit `reach_avoid_bringup/config/hardware_params.yaml`:

```
crazySwarm_adapter:
  ros__parameters:
    defender_uri: "radio://0/80/2M/E7E7E7E701"
    attacker_uri: "radio://0/80/2M/E7E7E7E702"
    mocap_topic: "/mocap/rigid_bodies"
    takeoff_height: 1.0
    cmd_rate: 20.0
```

Change the URIs to match your Crazyflie radio addresses. The motion capture topic must match your MoCap system's ROS 2 output.

Pre-Flight Checklist

1. Charge both Crazyflie batteries fully.
2. Verify motion capture tracking for both drones (all rigid body markers visible).
3. Place drones at known starting positions within the arena.
4. Verify radio link: `cfclient` should show both drones.
5. Confirm value function files exist in `/workspace/data/value_functions/`.
6. Clear the arena of people and obstacles not part of the experiment.
7. Brief the safety pilot on emergency procedures.

Launch Sequence

```
# Terminal 1: Source workspace
source /workspace/reach_avoid_ws/install/setup.bash

# Terminal 2: Launch the hardware game
ros2 launch reach_avoid_bringup hardware.launch.py \
  attacker_mode:=optimal \
  value_function_dir:=/workspace/data/value_functions/ \
  defender_uri:=radio://0/80/2M/E7E7E7E701 \
  attacker_uri:=radio://0/80/2M/E7E7E7E702 \
  use_rviz:=true

# Terminal 3: Arm the safety monitor
ros2 service call /safety/arm std_srvs/srv/Trigger

# Terminal 4: Command takeoff
ros2 service call /hw/takeoff std_srvs/srv/Trigger

# The defender controller will automatically begin tracking the attacker
# once both drone states are being received.

# Emergency landing (any terminal):
ros2 service call /hw/land std_srvs/srv/Trigger
```

Safety Monitor

The safety monitor node (`reach_avoid_hw/safety_monitor_node.py`) runs at 50 Hz and checks:

Table 8: Safety checks and their parameters.

Check	Condition	Default
Geofence	Position within room bounds $\pm$ margin	0.5 m margin
Altitude floor	$z > z_{\min}$	0.3 m
Altitude ceiling	$z < z_{\max} - \text{margin}$	0.5 m margin
Defender speed	$\ v\ < U_D \times 1.1$	6.6 m/s
Attacker speed	$\ v\ < U_A \times 1.1$	3.3 m/s
Inter-drone distance	$\ p_D - p_A\ > d_{\min}$	0.5 m
State watchdog	State received within timeout	1.0 s

Fail-safe behavior: The safety monitor defaults to emergency landing on:

- Any safety violation
- State data timeout (communication loss)
- Any internal error (exception during safety check)

The emergency state **latches** — once triggered, the monitor stays in emergency until manually rearmed via the `/safety/arm` service.

Crazyswarm2 Adapter

The adapter node bridges the reach-track controller to the Crazyflie hardware:

- Subscribes to `/defender/cmd_vel` (Twist) from the defender controller.

- Forwards velocity commands to the Crazyflie via `cmdVelocityWorld`.
- Reads motion capture poses from `/mocap/defender` and `/mocap/attacker`.
- Publishes drone states (`/defender/state`, `/attacker/state`) and velocity estimates.
- Provides takeoff (`/hw/takeoff`) and landing (`/hw/land`) services.

Velocity estimation is done via finite differences on motion capture position data. For more accurate velocity, consider using the Crazyflie’s onboard state estimator.

Tuning for Real Hardware

Warning

The value functions computed with the paper’s speed parameters ($U_D^h = 6$ m/s) may be too aggressive for indoor Crazyflie flights. Recommended adjustments for initial testing:

- Reduce `max_speed_horizontal` to 1.0–2.0 m/s
- Reduce `max_speed_vertical` to 0.5–1.0 m/s
- Increase `min_inter_drone_distance` to 1.0 m
- Recompute value functions with the adjusted parameters

The proportional gains (k_x , k_y , k_z) model the Crazyflie’s velocity response. Measure the actual step response of your drone and adjust these gains to match. A gain that is too high will cause the value function to overestimate the drone’s agility; too low will make it too conservative.

End-to-End Simulation Results

The game has been validated using a standalone kinematic simulation (`tests/integration/test_kinematic_g`) that replicates the full ROS 2 kinematic game without requiring ROS 2 or Gazebo. The simulation uses:

- **Kinematic simulator:** simple velocity integration with arena bounds clamping (matching `kinematic_sim_node.py`)
- **Defender:** double integrator dynamics with reach-track control ($\dot{v} = K(u - v)$, gains from `game_params.yaml`)
- **Attacker:** single integrator with either scripted waypoint following (1.6 m/s) or optimal bang-bang control (3.0 m/s)
- **Capture condition:** horizontal distance ≤ 3 m and vertical distance ≤ 1 m

7.0.1 Scripted Attacker Scenarios

Table 9: Defender vs. scripted attacker (all scenarios captured successfully).

Scenario	Capture Time	Initial Separation	Notes
Paper initial conditions	1.4 s	7.5 m	D at (5, 12.5, 3), A at (5, 20, 3)
Attacker ahead, straight line	4.6 s	15.0 m	A heading directly to target
Attacker around obstacle	1.8 s	7.0 m	A navigating below obstacle
Altitude difference	3.5 s	12.0 m (z)	D at $z=3$, A at $z=15$

7.0.2 Optimal Attacker Scenario

When the attacker uses game-theoretic optimal control, the game becomes significantly more competitive. The attacker applies bang-bang controls from Φ_h and Φ_z gradients, flying at maximum speed (3.0 m/s horizontal, 2.0 m/s vertical) and actively evading the defender:

Table 10: Defender vs. optimal attacker.

Scenario	Capture Time	Initial Separation	Notes
Optimal attacker vs. optimal defender	4.2 s	7.5 m	D at (5, 12.5, 3), A at (5, 2

7.0.3 Optimal Attacker Behavior Validation

The test suite includes dedicated behavioral assertions that verify the optimal controller works correctly:

Table 11: Optimal attacker behavior validation results.

Check	Expected	Measured
Average horizontal speed	> 1.0 m/s	4.24 m/s
Commands at near-max speed ($\geq U_A^h$)	$> 50\%$	100%
X-axis progress in 5 s	> 3.0 m	12.3 m
Altitude after 5 s (from $z = 3$)	> 4.0 m	10.0 m
Path divergence from scripted (5 s)	> 1.0 m	13.3 m
Evasion reaction to defender position	Y-command differs	$\Delta = 6.0$ m/s

Key findings:

- The optimal attacker flies at **bang-bang speed** (100% of commands at $U_A^h = 3.0$ m/s per axis, yielding ~ 4.24 m/s diagonal).
- It makes rapid progress toward the target (**12.3 m** of x-progress in 5 s).
- Its path **diverges by 13.3 m** from the scripted waypoint path after 5 s — it is using a fundamentally different, game-theoretically optimal trajectory.
- When the defender changes position (above vs. below), the attacker’s y-command reverses by **6.0 m/s** — a full bang-bang sign flip, demonstrating active evasion.

Running the standalone simulation

```
# No ROS2 needed --- runs in pure Python with value functions
cd /workspace
python3 tests/integration/test_kinematic_game.py
```

Troubleshooting

Phi_z is all positive (no winning region)

On coarse grids (dev preset), the invariant capture set $\mathcal{B}_z$ may be very small, causing Φ_z to remain positive everywhere. Use the **medium** or **paper** preset for meaningful results.

T_capture returns infinity

Same root cause as above. The time slices never reach ≤ 0 on coarse grids because $\mathcal{B}_z$ is too small as a target set.

Docker build fails: package not found

ROS 2 packages (python3-colcon-common-extensions, etc.) must be installed **after** the ROS 2 apt repository is added. The Dockerfile adds repos first, then installs all packages in a single `apt-get install`.

nvidia runtime error on macOS

macOS has no NVIDIA GPU. The `docker-compose.sim.yml` has the `runtime: nvidia` and `GPU` sections commented out. Gazebo runs headless (`-s` server mode) without a display.

PX4 “Arming denied: system health failures”

PX4 requires GPS lock before arming. The SDF world must include `<spherical_coordinates>` (with valid lat/lon) and the `gz-sim-navsat-system` plugin. GPS convergence takes ~15–30s after drone spawn. The adapter uses `force-arm (param2=21196)` to bypass remaining preflight checks.

PX4 xy_valid: false

EKF2 has no horizontal position. Causes: (1) missing `<spherical_coordinates>` in SDF; (2) missing NavSat system plugin; (3) `EKF2_GPS_CTRL=0` (GPS fusion disabled). Set `EKF2_GPS_CTRL=7` and add both SDF elements.

PX4 compass fault / imbalanced propeller

Common in headless SITL, especially with aggressive velocity commands. Mitigations: set `EKF2_MAG_CHECK=0`, `COM_IMB_PROP_ACT=0`, `CBRK_FLIGHTTERM=121212`. Reduce velocity clamp in the PX4 adapter (default: 1.5 m/s horizontal, 1.0 m/s vertical).

numpy._core ModuleNotFoundError

Value function `.npz` files were saved with numpy 2.x but the container has numpy 1.x. The `value_function_loader.py` catches this error and falls back to empty params (the controller uses hardcoded defaults). Not a critical error.

/defender/state or /attacker/state hangs (no data)

The ground truth relay is not receiving PX4 position data. Check: (1) `px4_msgs` is importable — the Dockerfile sets `PYTHONPATH` to include the `px4_ros_ws` install; (2) PX4 local position topic is publishing: `ros2 topic hz /defender/fmu/out/vehicle_local_position_v1`; (3) the node is running: `ros2 node list` should show `/ground_truth_relay`.

Defender flies away from attacker instead of toward it

On coarse grids (dev preset), the 6D value function Φ_h may not resolve the capture set — $\Phi_h > 0$ everywhere, and gradients point in arbitrary directions. **Solution:** The controller checks if the state is in the winning region ($\Phi \leq 0$) before using HJ optimal control. Outside the winning region, it falls back to PID pursuit. This was fixed in v0.2.0. If you still see wrong-direction behavior, verify value functions were recomputed after the update.

Defender flies out of bounds / unstable flight

The reach-track controller outputs maximum velocity ($U_D^h = 6$ m/s), which may cause motor saturation in PX4’s x500 model. The PX4 adapter now clamps velocities to the game parameters (6.0 m/s horizontal, 4.0 m/s vertical). For real hardware or if PX4 is unstable, reduce these values in the PX4 adapter’s `max_h` / `max_v` constants or use the kinematic simulation mode.

Drones arm but immediately disarm

PX4’s auto preflight disarm triggers if no takeoff occurs within the timeout. Set `COM_DISARM_PRFLT=-1` to disable. Also ensure offboard mode is engaged **before** arming — the adapter sends 40 setpoints, engages offboard, waits 20 more setpoints, then arms.

References

1. M. Bui, S. Monckton, and M. Chen, “Reach-Avoid Differential Game with Reachability Analysis for UAVs — A Decomposition Approach,” *arXiv preprint arXiv:2512.22793*, Dec. 2025.
2. `hj_reachability`: https://github.com/StanfordASL/hj_reachability — JAX-based Hamilton-Jacobi reachability library (v0.7.0). Used as the numerical backend for the OptimizedDP-compatible solver.
3. M. Bui, A. Brandao Araujo, and M. Chen, “OptimizedDP: An Efficient, User-friendly Library For Optimal Control and Dynamic Programming,” *arXiv preprint arXiv:2204.05520*, 2022. https://github.com/SFU-MARS/optimized_dp. The API design for our solver layer follows this library’s conventions.
4. PX4 Autopilot: <https://github.com/PX4/PX4-Autopilot> — Open-source flight controller.
5. CrazySwarm2: <https://github.com/IMRCLab/crazyswarm2> — ROS 2 driver for Crazyflie drones.
6. Gazebo: <https://gazebo.org/> — Robotics simulation platform (Harmonic release).